

Project Proposal: Smartphone Gait-Based Owner Verification

Efe Ozbal - S5996821
Sigurdur Haukur Birgisson - S5982960
Tijje Steeman - S5333644
Will Meeus - S5863708

May 1, 2026

1 Preliminary Domain Knowledge

During our preliminary research, we were especially interested in machine learning projects that make use of data already collected by everyday devices. Smartphones stood out as a strong example because they continuously record information from built-in sensors such as accelerometers and gyroscopes. This suggests that phones may support useful security or health-related applications without requiring extra hardware.

For our project, we want to apply this idea to a practical security problem: determining whether a phone remains under its owner's control while it is being carried by someone who is walking. This matters because theft or unauthorized access does not always happen in an obvious way. A phone might be taken and walked away with, or it might be picked up briefly for a nearby action such as making a payment and then returned. In either case, identifying only the exact moment the phone is taken is usually not enough.

Our main idea is that a person's gait may act as a behavioral fingerprint. A phone carried while walking records repeated motion patterns, and we believe these signals may contain user-specific information such as stride timing, acceleration structure, and frequency patterns. We found a previous study and repository that approached a related problem as a multiclass classification task, which suggests that different people may produce distinguishable motion signatures with high predictive accuracy.

Based on this, our project aims to model walking sequences from smartphone accelerometer and gyroscope data and map them into an embedding space. We then plan to use cosine similarity or another comparison method to estimate whether a new walking segment is likely to come from the phone's owner or from someone else. In other words,

the machine learning task is owner-versus-non-owner verification based on gait-related sensor data collected during walking.

2 Preliminary Data Exploration

The dataset we found on GitHub contains 3-axis gyroscope readings (GYR) and 3-axis accelerometer readings (ACC) over time, where Figure 1 shows the sensor axes. Figure 2a shows an example signal. This gives us the channels ACC_x , ACC_y , ACC_z , GYR_x , GYR_y , and GYR_z , each sampled at 50 Hz. As an attempt to filter out the high-frequency noise, we applied a low-pass filter to the raw signals, as shown in Figure 2b. Both raw and smoothed signals show a periodic pattern that is expected from the previous domain knowledge of the walking motion. Hence, we will extract features from frequency domain.

Other possible features we consider are the absolute acceleration, which is computed as $\sqrt{ACC_x^2 + ACC_y^2 + ACC_z^2}$, and absolute angular velocity, which is computed as $\sqrt{GYR_x^2 + GYR_y^2 + GYR_z^2}$. These features are expected to be informative because they capture the overall magnitude of acceleration and angular velocity regardless of direction, which can be indicative of the intensity and dynamics of the walking motion.

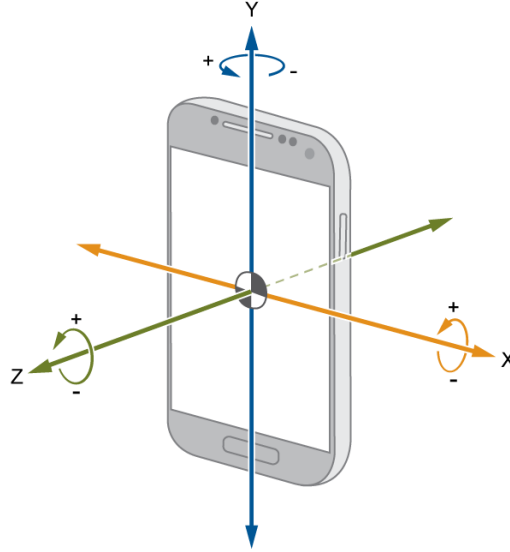


Figure 1: Sensor axes for accelerometer and gyroscope readings [1]

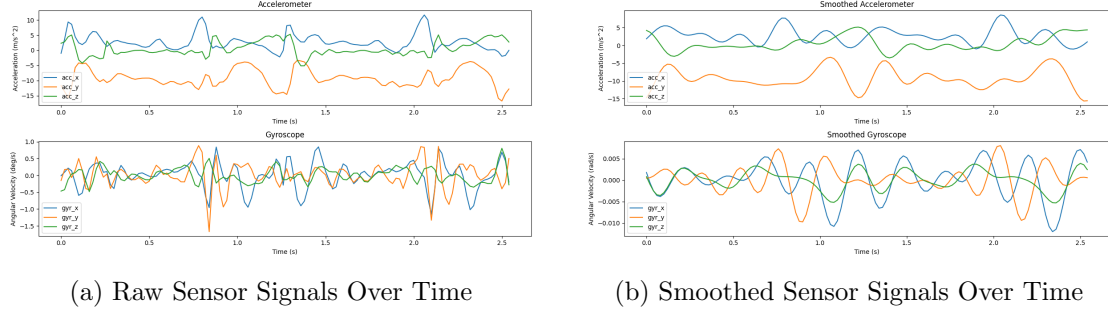


Figure 2: Time-series sensor signal examples

Since the task we are framing is a one-vs-rest verification problem, the class balance can be controlled during training by pairing each positive example with a negative comparison example.

We also computed the correlation matrix of the six sensor channels, which is shown in Figure 3. Even though majority of absolute value of channel correlations are close to zero (≤ 0.1), there can be seen a slight positive correlation of 0.355 between GYR_x and ACC_z . They are predicted to be correlated because both channels are influenced by the inward motion of the leg during stepping forward, where GYR_x captures the femur rotation and ACC_z captures the vertical acceleration from leg lifting.

Accompanying it, there is a negative correlation of -0.229 between GYR_z and ACC_x , which can be explained by the swinging motion of the leg during walking. Lastly, the correlation of 0.153 between ACC_z and ACC_y is believed to be, again, due to the inward motion of the leg during stepping forward. It should be noted that these correlations are weak and could be due to random chance.

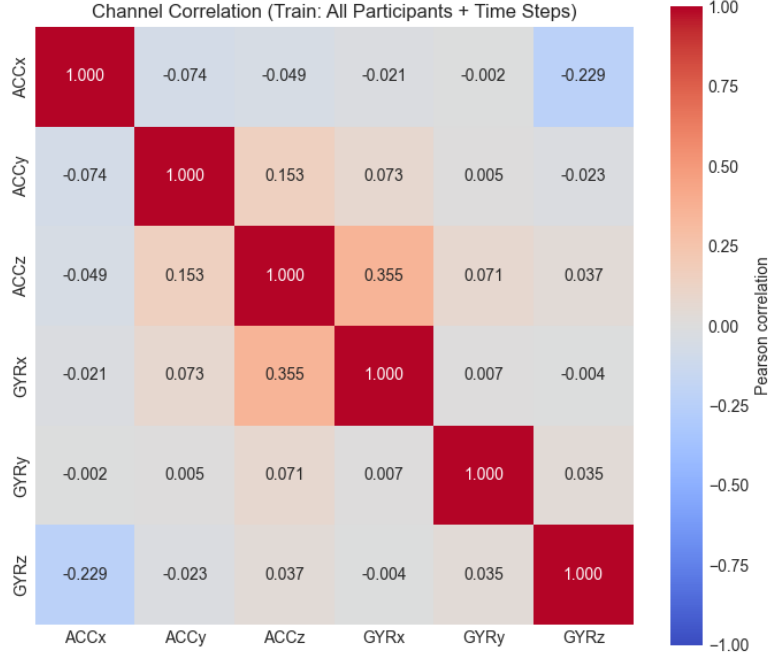


Figure 3: Correlation Matrix of Sensor Readings

There is no missing data or outliers in the dataset.

3 Proposed Preprocessing

We plan to:

- filter out obvious sensor anomalies or corrupted values if they appear;
- split the data by participant rather than randomly, so that the same person does not appear in both training and test data;
- avoid the original paper’s percentage-based split across each participant and instead enforce subject segregation;
- apply per-channel z-score normalization using statistics computed only on the training data;
- use subject-group K -fold cross-validation by dividing the 180 subjects into groups and folding by subject group;

- train on most subject groups, then for each test participant use a small amount of data to build the owner embedding and reserve the rest for testing;
- compare two preprocessing directions: using raw windows directly for deep learning, or doing feature engineering first;
- consider time-domain features such as mean, standard deviation, energy, turning points, and range, as well as frequency-domain features such as FFT band energy, dominant frequency, and spectral entropy;
- clip extreme values if we observe unrealistic sensor readings, although we have not seen obvious cases yet;
- consider time warping as a data augmentation method, although some temporal alignment is already handled through interpolation;
- decide whether to follow the paper’s compound-acceleration peak detection and two-step segmentation, or instead use broader movement windows if they capture more useful information; and
- select random adversarial or impostor subjects for each point so that each example is explicitly compared against someone it is not.

4 Proposed Model(s) and Baseline

We plan to:

- **Baseline:** FFT-based features with an SVM, or a similarly simple classical model.
- **Deep learning model:** still to be decided, but it will likely be trained on raw or lightly processed windows.
- **Loss:** triplet loss.
- **Model comparison:** accuracy, precision, recall, F1 score, confusion matrix, and K -fold comparison.
- **Hyperparameter search:** grid search.
- **Regularization techniques:** dropout, L2 regularization, early stopping based on F1, batch normalization, layer normalization, and Adam for optimization.
- **Hyperparameters to tune:** dropout, L2 strength, layer count, and node count.
- **Deployment goal:** compress the model to run on a phone through pruning, compression, quantization, or related methods.

- **Application goal:** build a web or phone app that records gyroscopic data live, feeds it to the model, and returns live predictions.

5 Proposed Evaluation

- **Metrics:** accuracy, precision, recall, F1 score, confusion matrix, K -fold comparison, and AUROC.
- **Evaluation of learned patterns:** for FFT-based features, we can inspect which frequencies are most useful for distinguishing between different pocket placements. We can visualize these features with a bar plot or heatmap to better understand what contributes to the model’s predictions.
- **Embedding visualization:** we want to visualize the embedding space to better understand what the model is using to separate users. If possible, we may also use methods similar to Grad-CAM or other interpretability tools.
- **Evaluation with new data:** we will record new data from a few volunteers who were not part of the original dataset. They will carry the phone in different pockets while walking, and we will use this data to evaluate the trained model. This should give us a better understanding of how well the model generalizes to new people and more realistic scenarios.

6 Model Usage

Our model could be used for:

- theft detection;
- suspicious activity detection;
- purchase confirmation based on recent motion behavior; and
- other lightweight security checks that require confirmation of identity.

Why: the system is relatively small, can run on-device, and has access to live motion data.

Who could use it: anyone with a smartphone, especially users who want an extra passive layer of security.

A user could download the app or access an API-based web app. If we can make the model small enough, we may be able to deploy it directly in the browser or as a lightweight local download. There would first be a short enrollment period where the user walks for a few minutes to create an identification embedding. After that, live data would be streamed in, interpolated and resampled to handle jitter, and then compared through cosine similarity or another comparison method against the known owner embedding.

Most of the time the system would remain idle. If it makes a prediction that suggests a possible threat, the app could trigger an alert, message, or notification on another device.

The final output is essentially a yes/no decision, so the user-facing result would likely be a notification on another device. To support that, we would need SMS, email, bot-based messaging, or a server that can receive the signal and send a ping to another device.

7 Risk Assessment

One major risk is the use of live streaming data, since this is sensitive information. For that reason, we would prefer to keep the system on-device rather than stream sensor data to a server.

We may also face a trade-off between model size and accuracy, and we may need to learn deployment techniques that are specific to this use case. To prepare for that, we want to look early at projects such as the web **fake beer** drinking app to understand how live streaming sensor data is handled, and we may also need to explore mobile development.

The project is likely to be successful in terms of model accuracy, since the task has already been shown to be possible with high accuracy. What is less certain is how feasible the approach will be once compression and deployment constraints are added.

The main societal risk is misuse: the system could be used to detect, follow, or identify someone through their embedding, which would effectively leak biometric information. There are also user risks because this system would be an added security layer, and users might blame the model if it fails in a sensitive situation. In a real deployment, the model would need to achieve a very high level of reliability before it could be trusted.

8 Individual Learning Outcomes

Will: I would like to learn more about model compression as a general practice and about the nuance of deploying such a model. This connects to my interest in neuromorphic computing, because smaller neural networks are easier to simulate in that setting and can provide useful insight into how these ideas might be implemented in circuitry.

Haukur:

Tijje:

Efe:

9 Relation to Grading Specifications

In previous sections, we have already described how we will be pre-processing the data, conducting cross-validation, and optimizing hyperparameters.

For the evaluation of over/underfitting with regularization, we already mentioned that we will be using regularization techniques as well as early stopping to prevent overfitting.

In terms of novelty, we believe our project is novel as no previous work on the real world application of gait-based owner verification has been done. To this end, we aim to compress the model to be able to run on a smartphone.

10 Timeline

The first week corresponds to the week of 20-26 April.

Task	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8
Project Search/Selection								
Literature Search								
Proposal Writing								
Data Exploration								
Preprocessing								
Training baseline model								
Training main model								
Evaluation								
Model Deployment								
Implementing Extras								
Report Writing								

Table 1: Proposed timeline for the project.

References

References

- [1] MathWorks. Gyroscope block [digital image]. MathWorks Help Center, 2014. In *Gyroscope – Measure rotational speed around X, Y, and Z axes in rad/s*. Retrieved April 30, 2026, from https://nl.mathworks.com/help/simulink/supportpkg/android_ref/gyroscope.html.